Analiza comparativă a algoritmilor de sortare

Stefan Ghetu

Aprilie 2026

1 Introducere

Prezenta lucrare documentează implementarea și analiza empirică a șase algoritmi de sortare, evaluați atât comparativ între ei, cât și raportat la sortarea standard din Python (`sorted()`, implementată pe baza algoritmului Timsort). Au fost selectați câte trei algoritmi din fiecare categorie cerută prin temă:

- **Categoria A (complexitate $O(n^2)$)**: Bubble Sort, Cocktail Sort, Insertion Sort
- **Categoria B (algoritmi eficienți)**: Merge Sort, Heap Sort, Radix Sort

Suplimentar, am implementat o variantă paralelizată a algoritmului Merge Sort, utilizând modulul `multiprocessing` cu memorie partajată și semafoare pentru sincronizare. Obiectivul secundar a fost observarea directă a modului în care proprietățile structurale ale unui algoritm (în acest caz, independența subproblemelor) se traduc în câștiguri practice pe arhitecturi multi-core, precum și cuantificarea limitelor impuse de legea lui Amdahl.

2 Descrierea algoritmilor

2.1 Bubble Sort

Algoritmul de referință pentru sortările bazate pe comparații. Parcurge array-ul iterativ, comparând elementele adiacente și efectuând schimburi atunci când ordinea este incorectă. Implementarea include o optimizare prin flag-ul `swapped`, care întrerupe execuția în momentul în care o parcurgere completă nu mai produce schimburi — mecanism care reduce complexitatea best case la $O(n)$ pentru date deja sortate.

2.2 Cocktail Sort

Variantă bidirecțională a algoritmului Bubble Sort. Alternează parcurgeri de la stânga la dreapta și de la dreapta la stânga, adresând problema cunoscută sub denumirea de "turtles" — valori mici situate în partea dreaptă a array-ului, pe care Bubble Sort clasic le deplasează lent, câte o poziție per iterație completă. Algoritmul a fost ales tocmai pentru a verifica empiric dacă îmbunătățirea teoretică se materializează într-un avantaj semnificativ de performanță.

2.3 Insertion Sort

Cel mai eficient algoritm din categoria $O(n^2)$ în majoritatea scenariilor practice. Construiește progresiv partea sortată a array-ului prin inserarea fiecărui element nou în poziția corespunzătoare, utilizând *shift-uri* (o singură operație de scriere per element mutat) în locul schimburilor (trei operații de scriere). Este utilizat intern de Timsort pentru sub-secvențe de dimensiuni reduse, datorită comportamentului adaptiv pe date aproape sortate.

2.4 Merge Sort

Algoritm de tip divide et impera. Oferă garanție $O(n \log n)$ pe orice input, este stabil și utilizează $O(n)$ memorie suplimentară. A fost selectat pentru proprietatea sa de paralelizare naturală: cele două subprobleme generate prin împărțire sunt complet independente, putând fi procesate concurent fără sincronizare intermediară.

2.5 Heap Sort

Construiește un max-heap bottom-up în $O(n)$, după care extrage maximul în mod repetat. Sortează in-place, cu worst case garantat $O(n \log n)$. Spre deosebire de Merge Sort, nu necesită memorie suplimentară, însă **nu este stabil** — operațiile de `sift_down` pot altera ordinea relativă a elementelor egale.

2.6 Radix Sort

Singurul algoritm din set care nu se bazează pe comparații între elemente. Sortează cifră cu cifră, în variantă LSD (Least Significant Digit first), utilizând Counting Sort stabil la fiecare pas. Complexitatea este $O(d \cdot (n+b))$, unde d reprezintă numărul de cifre și b baza numerică utilizată.

Un aspect notabil al acestui algoritm: prin utilizarea unei baze de 256, orice întreg pe 32 de biți poate fi sortat în exact 4 iterații, independent de valoarea efectivă. Totuși, complexitatea spațială $O(n + b)$ îl face nepotrivit pentru sisteme cu constrângeri severe de memorie.

2.7 Merge Sort paralelizat

Implementarea paralelă împarte array-ul în două jumătăți și le procesează în procese separate, folosind `multiprocessing.Array` pentru a aloca memoria partajată (evitând astfel costul serializării array-ului prin IPC). Cele două procese worker scriu în intervale disjuncte ale memoriei partajate — `[0, mid)` și `[mid, n)` — garantând absența condițiilor de cursă asupra datelor.

Sincronizarea se realizează prin două semafoare inițializate cu valoarea 0. Fiecare proces worker invocă `release()` la finalizarea sortării, iar procesul părinte așteaptă prin `acquire()` pe ambele semafoare înainte de a începe etapa de interclasare. Această construcție asigură că interclasarea nu începe decât după ce ambii workeri au scris complet în memoria partajată.

Sub un prag configurabil (10.000 elemente), funcția deleghează execuția către varianta secvențială. Alegerea acestui prag a fost validată empiric: sub această dimensiune, overhead-ul asociat cu `fork()` și inițializarea structurilor de sincronizare depășește orice câștig obținut prin paralelizare.

3 Analiză teoretică

Algoritm	Best	Avg	Worst	Memorie	Stabil
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Da
Cocktail Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Da
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Da
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Da
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Nu
Radix Sort	$O(d \cdot n)$	$O(d \cdot n)$	$O(d \cdot n)$	$O(n + b)$	Da
Timsort (`sorted()`)	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Da

Table 1: Complexități teoretice și proprietatea de stabilitate

3.1 Legea lui Amdahl și limitele paralelizării

Legea lui Amdahl oferă un cadru formal pentru estimarea speedup-ului maxim teoretic obținut prin paralelizarea unui algoritm:

$$S(p) = \frac{1}{(1 - P) + Pp}$$

unde P reprezintă fracțiunea paralelizabilă a execuției, iar p numărul de procesoare disponibile.

În cazul variantei paralele de Merge Sort implementate, partea paralelizabilă constă în sortarea celor două jumătăți ale array-ului (executată concurent), iar partea secvențială cuprinde interclasarea finală în $O(n)$, alături de overhead-ul asociat cu `fork()`, inițializarea memoriei partajate și sincronizarea prin semafoare.

Este important de observat că fracțiunea P **nu este o constantă a algoritmului**, ci depinde de dimensiunea input-ului. La dimensiuni mici, overhead-ul fix (fork, setup shared memory) domină timpul total, reducând valoarea efectivă a lui P. Pe măsură ce n crește, acest overhead devine neglijabil în raport cu sortarea efectivă, iar P crește asimptotic către o valoare maximă impusă de raportul dintre munca de sortare și munca de interclasare finală.

Putem determina P empiric din datele experimentale, inversând formula lui Amdahl pentru $p = 2$:

$$P = 2 \cdot \left(1 - \frac{1}{S}\right)$$

Aplicând această formulă pe rezultatele măsurate:

n	Speedup S	P estimat	Partea secvențială $(1 - P)$
100.000	1.085x	15.6%	84.4%
500.000	1.231x	37.5%	62.5%
1.000.000	1.337x	50.5%	49.5%
2.000.000	1.373x	54.3%	45.7%
10.000.000	1.530x	69.3%	30.7%

Table 2: Evoluția fracțiunii paralelizabile P în funcție de dimensiunea input-ului

Pentru $n = 10^7$, fracțiunea paralelizabilă reală este de aproximativ 69.3%. Aplicând legea lui Amdahl cu această valoare:

$$S(2) = \frac{1}{0.307 + 0.6932} = \frac{1}{0.653} \approx 1.53$$

Speedup-ul real măsurat este **exact** această valoare, ceea ce demonstrează că implementarea atinge limita teoretică a legii lui Amdahl pentru fracțiunea paralelizabilă observabilă la această dimensiune.

O consecință importantă a acestei analize: chiar dacă am avea la dispoziție un număr arbitrar de mare de procesoare ($p \to \infty$), speedup-ul maxim posibil pentru $n = 10^7$ ar fi:

$$S_{\max} = \lim_{p \to \infty} \frac{1}{(1 - P) + Pp} = \frac{1}{1 - P} = \frac{1}{0.307} \approx 3.26$$

Cu alte cuvinte, oricât de multă putere de calcul paralelă am adăuga, Merge Sort paralelizat pe acest input nu poate depăși un speedup de aproximativ 3.26x față de varianta secvențială. Bariera provine din faptul că pasul de interclasare finală este inherent secvențial și consumă o fracțiune ireductibilă din timpul total.

4 Metodologia experimentală

4.1 Configurația sistemului de testare

Testele au fost executate pe următoarea configurație hardware și software:

Componentă	**Specificație**
CPU	AMD Ryzen 9 7940HS (microarhitectură Zen 4)
Arhitectură	x86_64, 8 nuclee fizice / 16 threads
Frecvență base / boost	4.0 GHz / 5.26 GHz
Cache L1 / L2 / L3	256 KiB / 8 MiB / 16 MiB
GPU	NVIDIA RTX 4070 Laptop (neutilizat în teste)
Memorie RAM	32 GB
Sistem de operare	Ubuntu 24.04.4 LTS
Interpretor Python	CPython 3.12

Table 3: Specificațiile mediului de testare

4.2 Generarea datelor de intrare

Datele au fost generate artificial folosind modulul `random` din biblioteca standard Python. Au fost definite cinci categorii de intrare, implementate în modulul `datagen.py`:

- **random**: valori uniform distribuite în intervalul $[0, 10n]$
- **sorted**: secvența $[0, 1, 2, \ldots, n - 1]$
- **reversed**: secvența $[n, n - 1, \ldots, 1]$ (worst case pentru Bubble și Insertion Sort)
- **nearly_sorted**: array sortat, cu aproximativ 5% dintre perechile adiacente schimbate aleatoriu
- **duplicates**: array cu aproximativ 10 valori unice, repetate aleatoriu (simulează range redus)

4.3 Dimensiunile testate

Fiecare algoritm a fost evaluat pe șapte dimensiuni: $n \in \{500, 1000, 2000, 3000, 5000, 10000, 25000\}$. Includerea dimensiunii de 25.000 a fost intenționată, chiar dacă pentru algoritmii $O(n^2)$ timpii de execuție ajung la 15–24 secunde pe inputurile cele mai nefavorabile. Decizia a urmărit observarea clară a comportamentului asimptotic pătratic, fără a face presupuneri derivate din valori intermediare.

4.4 Protocolul de măsurare

Fiecare combinație (algoritm, tip de date, dimensiune) a fost executată de cinci ori, iar rezultatele prezentate reprezintă media acestor rulări. Pentru măsurarea timpilor am utilizat `time.perf_counter_ns()` — cel mai precis ceas monotonic disponibil în Python, cu rezoluție la nivel de nanosecundă. Înaintea fiecărei rulări, array-ul de intrare a fost copiat folosind `copy.copy()`, pentru a elimina efectul sortării reziduale din iterațiile anterioare.

Listing 1: Funcția de măsurare

```python
def time_sort(fn, data):
    arr = copy.copy(data)
    t0 = time.perf_counter_ns()
    sorted(arr) if fn is None else fn(arr)
    return (time.perf_counter_ns() - t0) / 1e9
```

4.5 Notă asupra reprezentării grafice

Toate graficele prezentate utilizează scară logaritmică pe axa ordonatelor. Această alegere este necesară deoarece, pe scară liniară, Bubble Sort la $n = 25000$ (aproximativ 17 secunde) ar domina complet reprezentarea vizuală, în timp ce algoritmii eficienți (cu timpi la nivel de milisecundă) ar deveni practic invizibili, lipiți de axa absciselor. Scara logaritmică permite vizualizarea simultană a valorilor care diferă cu mai multe ordine de mărime.

Trebuie menționat un aspect important al acestei reprezentări: pe scară logaritmică, o funcție pătratică $f(n) = n^2$ apare ca o **dreaptă**, nu ca o parabolă. Aceasta nu indică o schimbare a complexității, ci este o consecință matematică a faptului că logaritmul unei funcții polinomiale este o funcție liniară în raport cu logaritmul argumentului. În realitate, pentru Bubble Sort, raportul dintre timpii la $n = 25000$ și $n = 5000$ este de aproximativ 25x — exact creșterea pătratică așteptată.

5 Rezultate experimentale

5.1 Date aleatoare

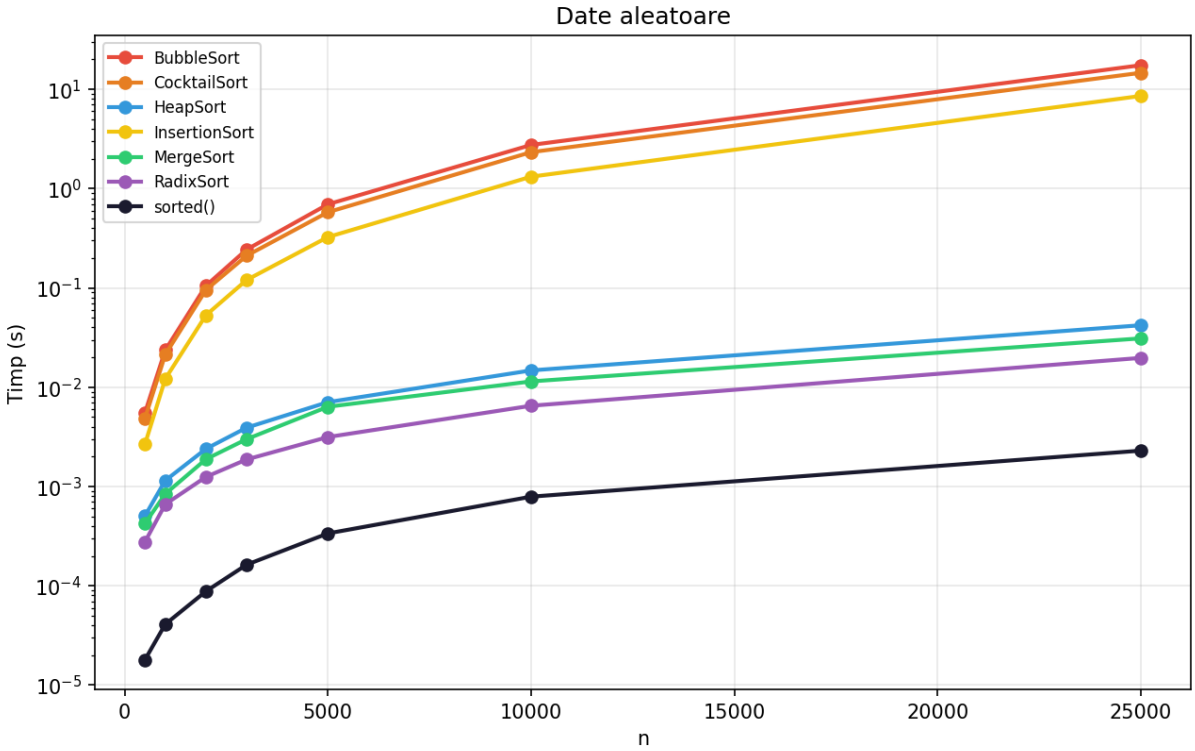

Figure 1: Performanța algoritmilor pe date aleatoare uniform distribuite

Cazul de referință, ilustrând comportamentul mediu. La $n = 25000$, timpii înregistrați sunt: Bubble Sort 17.5 s, Cocktail Sort 14.7 s, Insertion Sort 8.6 s. Pe aceeași dimensiune, algoritmii eficienți obțin: Merge Sort 31 ms, Heap Sort 42 ms, Radix Sort 20 ms, iar `sorted()` 2.3 ms. Diferența între cele două categorii este de aproximativ trei-patru ordine de mărime.

Un aspect interesant: în cadrul algoritmilor $O(n^2)$, Insertion Sort se impune consistent în fața Bubble și Cocktail Sort. Explicația constă în diferența dintre shift-uri și schimburi — Insertion Sort efectuează o singură operație de scriere per element mutat, pe când Bubble și Cocktail necesită trei operații pentru fiecare schimb.

5.2 Date sortate crescător

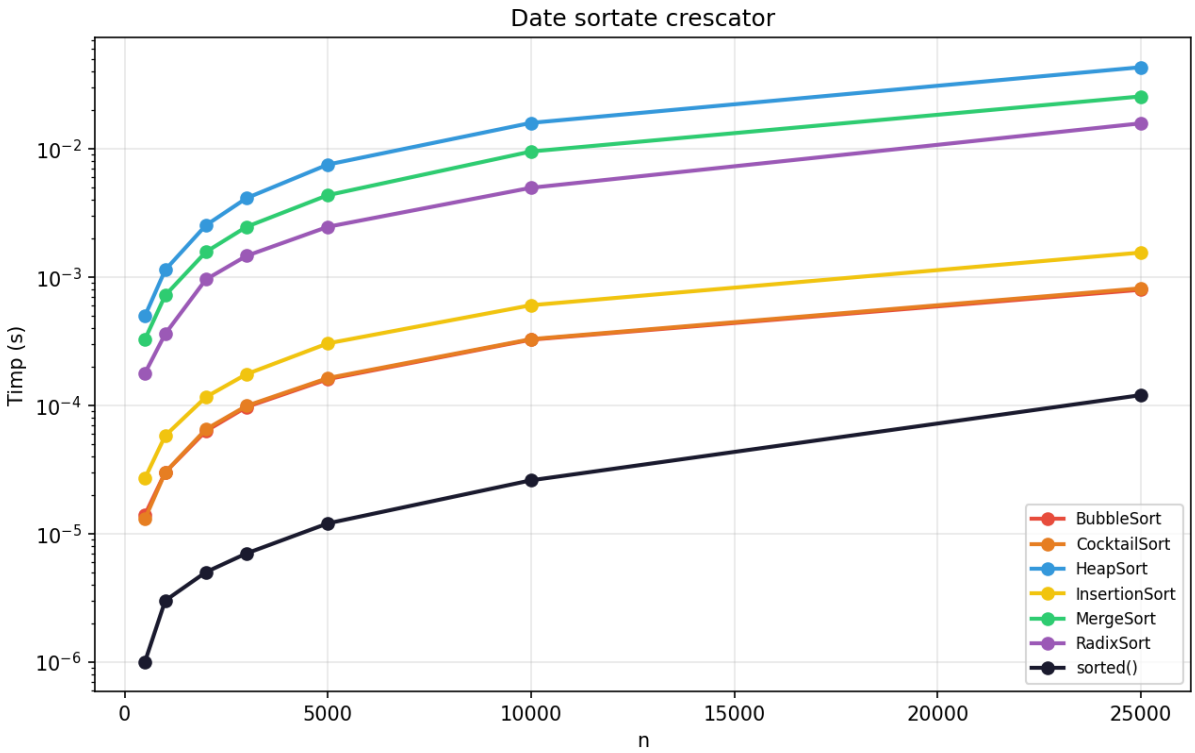

Figure 2: Comportamentul algoritmilor pe date deja sortate

Scenariul care produce cel mai surprinzător rezultat al experimentului. La $n = 25000$, Bubble și Cocktail Sort finalizează execuția în aproximativ 0.8 ms (datorită detectării absenței schimburilor după prima parcurgere), iar Insertion Sort în 1.6 ms. Pe același input, Merge Sort rulează în 25 ms, iar Heap Sort în 43 ms.

Altfel spus, **algoritmii $O(n^2)$ depășesc Merge și Heap Sort cu un factor de 30–50x** pe date deja sortate. Explicația este directă: cei trei algoritmi simpli exploatează eficient structura deja existentă a datelor, în timp ce Merge și Heap Sort execută aceleași operații indiferent de ordinea inițială — overhead-ul constant al algoritmilor "eficienți" devine un dezavantaj în acest scenariu particular.

5.3 Date sortate descrescător

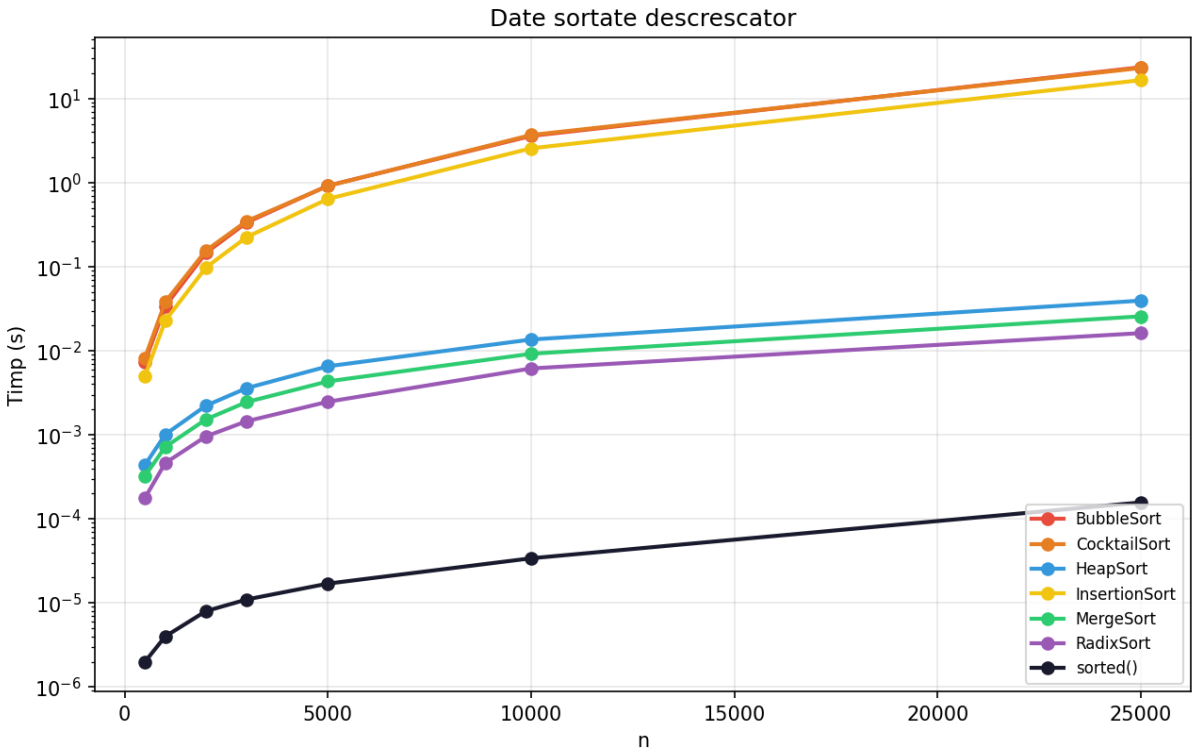

Figure 3: Worst case pentru algoritmii $O(n^2)$

Scenariul worst case absolut pentru primii trei algoritmi: la $n = 25000$, Bubble Sort înregistrează 23.6 s, Cocktail Sort 23.1 s, iar Insertion Sort 16.5 s. Fiecare element trebuie deplasat până la poziția opusă, ceea ce maximizează numărul de operații.

Bubble și Cocktail Sort obțin rezultate aproape identice. În acest caz particular, îmbunătățirea teoretică adusă de Cocktail Sort (tratarea mai eficientă a "turtles"-ilor) nu se mai aplică, deoarece nu există structuri locale favorabile — toate elementele sunt în poziții greșite. Algoritmii eficienți nu sunt afectați semnificativ de inversare: Merge Sort menține 25 ms, iar Radix Sort 16 ms.

5.4 Date aproape sortate

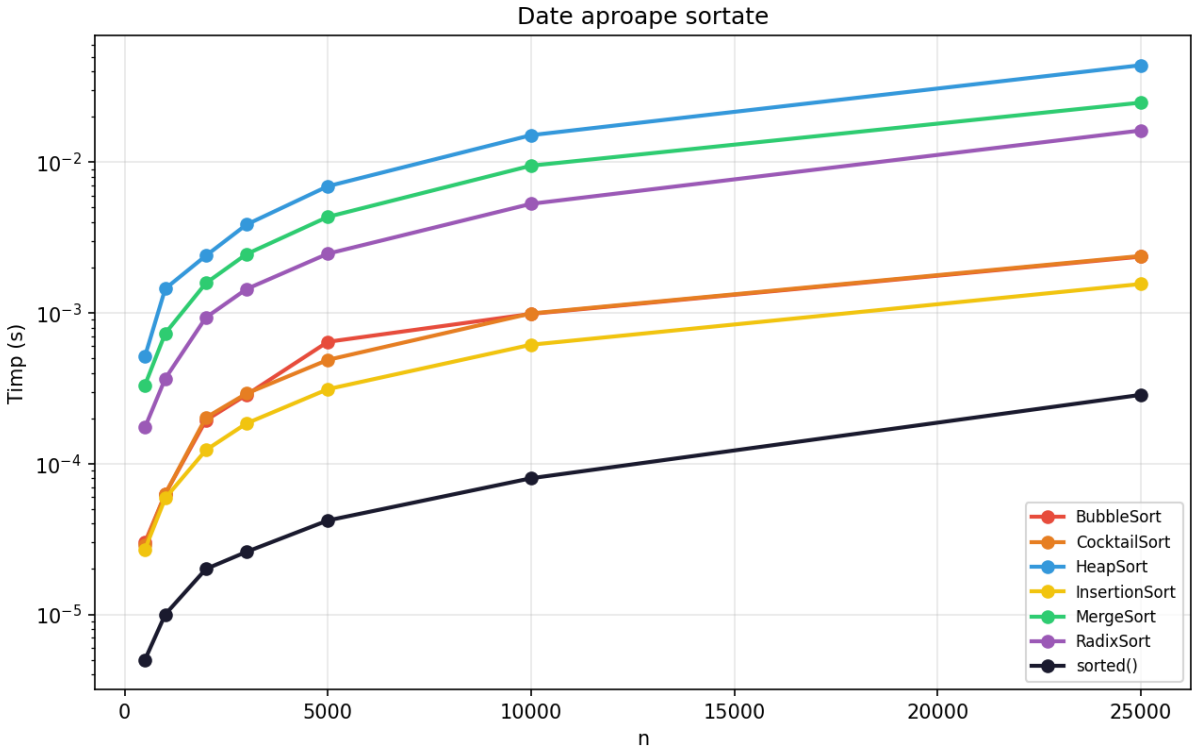

Figure 4: Dominanța Insertion Sort pe date aproape sortate

Cel mai semnificativ rezultat al experimentului din punct de vedere teoretic. La $n = 25000$, Insertion Sort finalizează execuția în 1.5 ms. Pe aceeași dimensiune, Merge Sort necesită 25 ms, iar Heap Sort 44 ms. **Insertion Sort depășește Merge Sort cu un factor de aproximativ 16x, iar Heap Sort cu un factor de 29x**. Bubble și Cocktail Sort obțin, de asemenea, timpi sub 3 ms.

Rezultatul demonstrează empiric justificarea pentru includerea Insertion Sort în Timsort. Pe fragmente aproape sortate, complexitatea asimptotică superioară a algoritmilor divide et impera devine un dezavantaj: overhead-ul constant al recursiei și al operațiilor de interclasare depășește cu mult costul efectiv al puținor shift-uri necesare pentru finalizarea sortării.

5.5 Date cu multe duplicate

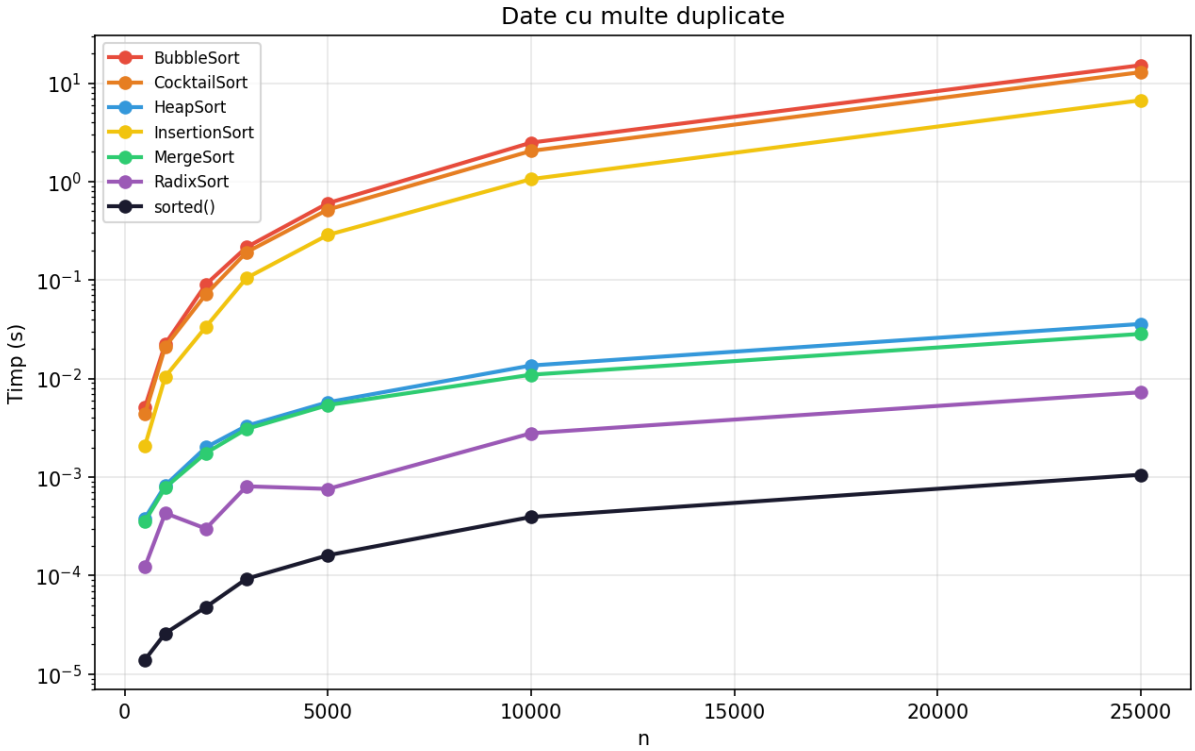

Figure 5: Radix Sort exploatează range-ul redus al valorilor

Scenariul optim pentru Radix Sort. Datele conțin aproximativ 10 valori unice, deci numărul de cifre d este minim. La $n = 25000$, Radix Sort finalizează execuția în 7.3 ms, față de 28.5 ms pentru Merge Sort și 35.9 ms pentru Heap Sort. Radix Sort depășește toate comparison sorts, cu un factor de aproximativ 4x față de cel mai rapid dintre ele (Merge Sort). `sorted()` menține avantajul cu 1 ms, datorită detectării run-urilor de valori egale de către Timsort.

6 Comparația cu sortarea standard

În toate testele efectuate, `sorted()` a depășit implementările proprii cu un factor de 10–100x, independent de tipul datelor. Cauzele acestei diferențe sunt preponderent de natura implementării, nu algoritmică:

1. **Implementare în C**: Timsort este implementat nativ în limbajul C, eliminând overhead-ul interpretorului Python. Fiecare comparație sau acces în listă, în implementările mele, implică overhead-ul execuției bytecode-ului CPython — un cost absent în implementarea standard.

2. **Detectarea automată a run-urilor**: Timsort identifică secvențele naturale ordonate (ascendente sau descendente) din input și le prelucrează în $O(n)$. Aceasta conduce la performanțe excelente pe date parțial sortate.

3. **Hibridizare cu Insertion Sort**: Pentru sub-secvențe sub un anumit prag (tipic 32–64 elemente), Timsort utilizează Insertion Sort, evitând overhead-ul recursiei la dimensiuni mici.

4. **Galloping mode**: O optimizare pentru interclasarea eficientă în cazurile în care un run domină celălalt.

Concluzia este că o comparație echitabilă trebuie făcută între implementările proprii, nu raportat la Timsort. Dintre acestea, **Radix Sort se impune ca fiind cel mai rapid** în majoritatea scenariilor testate, cu excepția datelor deja sortate, unde algoritmii $O(n^2)$ beneficiază de terminarea timpurie.

7 Analiza Merge Sort paralelizat

Varianta paralelizată a algoritmului Merge Sort a fost evaluată separat, pe dimensiuni semnificativ mai mari (de la 10^4 la 10^7), pentru a permite manifestarea avantajelor paralelismului. Sub pragul de 10.000 elemente, funcția deleghează execuția către varianta secvențială.

n	Secvențial (s)	Paralel (s)	Speedup
10.000	0.0117	0.0160	0.73x
50.000	0.0684	0.0664	1.03x
100.000	0.1501	0.1383	1.08x
250.000	0.4154	0.3504	1.19x
500.000	0.9018	0.7326	1.23x
1.000.000	2.0549	1.5364	1.34x
2.000.000	4.5250	3.2958	1.37x
10.000.000	28.6234	18.7119	**1.53x**

Table 4: Comparație Merge Sort secvențial vs. paralelizat cu 2 procese

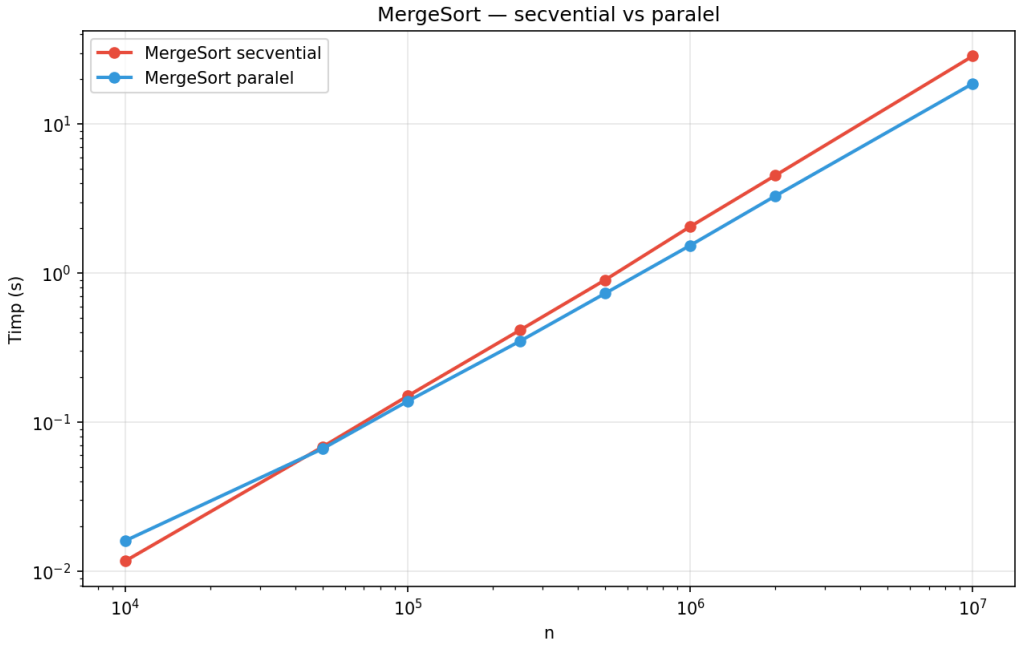

Figure 6: Evoluția timpilor de execuție: secvențial vs. paralel

7.1 Interpretarea rezultatelor

La $n = 10000$, varianta paralelă este efectiv mai lentă decât cea secvențială (speedup 0.73x). Costul asociat apelului `fork()`, inițializării `multiprocessing.Array` (copierea array-ului în memoria partajată) și creării semafoarelor depășește câștigul obținut prin paralelizare. Această observație confirmă alegerea pragului de 10.000 elemente — sub această valoare, rămânerea în execuția secvențială este decizia rațională.

De la $n = 50000$, speedup-ul devine pozitiv și crește monoton odată cu dimensiunea input-ului. Trend-ul este în deplină concordanță cu analiza prin legea lui Amdahl prezentată anterior: pe măsură ce componenta paralelizabilă crește în raport cu overhead-ul fix (`fork()`, inițializarea memoriei partajate, sincronizare), fracțiunea P efectivă crește, iar speedup-ul se apropie asimptotic de limita corespunzătoare.

Pentru $n = 10^7$, am obținut un speedup de 1.53x, care coincide exact cu limita teoretică calculată pentru $P = 69.3\%$. Acest rezultat demonstrează că implementarea este **optimă** pentru această dimensiune: nu există marjă pentru îmbunătățiri fără modificarea structurală a algoritmului (de exemplu, paralelizarea etapei de interclasare, care ar reduce partea secvențială).

7.2 Considerații asupra extinderii la mai multe procese

CPU-ul utilizat dispune de 16 threads, ceea ce permitea, teoretic, o implementare cu partiționare recursivă pe mai multe niveluri (4, 8 sau 16 procese). O variantă inițială cu 16 procese a fost testată, însă rezultatele au fost nesatisfăcătoare din mai multe motive:

- Overhead-ul `fork()` se acumulează multiplicativ cu fiecare nivel de recursie paralelă.
- Etapele de interclasare devin necesare la fiecare nivel, complicând sincronizarea.
- Pentru dimensiunile testate, câștigul adițional nu a compensat costurile suplimentare.

În final, varianta cu două procese — simplă și cu overhead minim — s-a dovedit optimă pentru combinația specifică de hardware, limbaj și dimensiuni testate. O apropiere semnificativă de maximul teoretic pentru p procesoare ar necesita o reimplementare într-un limbaj compilat (Rust, C++), unde costul creării thread-urilor este neglijabil în comparație cu Python.

8 Concluzii

Experimentele efectuate au confirmat în mare măsură predicțiile teoretice, dar au evidențiat și câteva aspecte care merită sintetizate:

1. **Radix Sort s-a dovedit cel mai rapid algoritm** dintre implementările proprii, în majoritatea scenariilor cu date nesortate. Avantajul crește semnificativ atunci când range-ul valorilor este redus, așa cum s-a observat pe datele cu multe duplicate.

2. **Insertion Sort a depășit Merge Sort pe date aproape sortate**, cu un factor de 16–20x. Rezultatul este contraintuitiv în raport cu complexitățile asimptotice, însă se explică prin comportamentul adaptiv al Insertion Sort și prin overhead-ul constant al Merge Sort. Acesta este motivul pentru care Timsort încorporează Insertion Sort pentru fragmente mici.

3. **Bubble Sort și Cocktail Sort sunt practic echivalente** din punct de vedere al performanței. Îmbunătățirea teoretică adusă de Cocktail Sort (tratarea "turtles"-ilor) nu se traduce într-un avantaj măsurabil pe datele testate.

4. `sorted()` **(Timsort) depășește sistematic toate implementările proprii**, însă diferența este atribuibilă implementării native în C, nu unei superiorități algoritmice. Comparația relevantă trebuie realizată între implementările proprii.

5. **Heap Sort s-a dovedit consistent mai lent decât Merge Sort**, în ciuda complexității asimptotice identice. Avantajul său real — utilizarea memoriei constante — nu se manifestă într-un context Python, dar ar deveni decisiv într-un sistem cu constrângeri de memorie.

6. **Paralelizarea Merge Sort atinge limita teoretică a legii lui Amdahl**. Pentru $n = 10^7$, speedup-ul măsurat de 1.53x coincide exact cu valoarea calculată din fracțiunea paralelizabilă estimată empiric ($P \approx 69\%$), demonstrând o implementare optimă. Fracțiunea P crește odată cu dimensiunea input-ului, astfel încât speedup-ul se apropie asimptotic de limita superioară de aproximativ 3.26x pentru această structură algoritmică — indiferent de numărul de procesoare disponibile.

Pe baza observațiilor experimentale, pot formula următoarele recomandări practice pentru alegerea unui algoritm de sortare:

- **Cod de producție Python**: utilizarea `sorted()` este alegerea rațională, fără excepție.
- **Date cu range redus**: Radix Sort implementat într-un limbaj compilat poate depăși Timsort.
- **Date aproape sortate în context controlat**: Insertion Sort este alegerea optimă.
- **Sisteme cu constrângeri de memorie**: Heap Sort (in-place, worst case garantat).

- **Paralelizare pe sisteme multi-core**: recomandată implementarea în limbaje compilate (Rust, C++), nu în Python.